SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	**192425199**
Student Name	**A. Mohith Krishna**

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
  FOR j ← i + 1 TO n − 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller = Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i − 1 DO
    Audit(Request[j])        // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Source Code:

```
import math
# ShopSphere Scalability Audit
N = 1_000_000
OPS_PER_SECOND = 100_000_000
DEADLINE = 600
# (a) Understand the Problem
def question_a():
    print("\n========== (a) UNDERSTAND THE PROBLEM ==========")
    print("M1: Pair comparisons - Theta(n^2)")
    print("M2: Request audits - Theta(n)")
    print("S1: Linear search - O(n)")
    print("S2: Binary search - Theta(log n)")
    print("M4 Current: 4T(n/2) + n - Theta(n^2)")
    print("M4 Proposed: 2T(n/2) + n - Theta(n log n)")
    print("M5: 2T(n/2) + n log n - Theta(n log^2 n)")
# (b) Iterative Modules
def m1_comparisons(n):
    return n * (n - 1) // 2
def m2_audits(n):
    i = 1
    count = 0
    while i < n:
        count += i
        i *= 2
    return count
def question_b():
    print("\n========== (b) ITERATIVE MODULES ==========")
    print("M1 comparisons = n(n-1)/2")
    print("M1 time complexity = Theta(n^2)")
    print("M1 auxiliary space = Theta(1)")
    print(f"M1 comparisons at n={N:,}: {m1_comparisons(N):,}")
```

```
    print("\nM2 audit operations form a geometric series.")
    print("M2 time complexity = Theta(n), not Theta(n log n)")
    print("M2 auxiliary space = Theta(1)")
    print("M2 audits at n=1024:", m2_audits(1024))
    print(f"M2 audits at n={N:,}: {m2_audits(N):,}")
# (c) Recurrence by Backward Substitution
def binary_search_comparisons(n):
    count = 0
    while n > 0:
        count += 1
        n //= 2
    return count
def question_c():
    print("\n========== (c) BACKWARD SUBSTITUTION ==========")
    print("T(n) = T(n/2) + 1")
    print("T(n) = T(n/4) + 2")
    print("T(n) = T(n/8) + 3")
    print("After k steps: T(n) = T(n/2^k) + k")
    print("Stop when n/2^k = 1, so k = log2(n)")
    print("Since T(1) = 1, T(n) = 1 + log2(n)")
    print("Time complexity = Theta(log n)")
    print("Iterative binary search space = Theta(1)")
    print("Recursive binary search space = Theta(log n)")
    print("Comparisons for one search at n=1,000,000:",
          binary_search_comparisons(N))
# (d) Master Theorem and Recurrence Estimates
def m4_current(n):
    # T(n) = 4T(n/2) + n, assuming T(1) = 1
    return 2 * n * n - n
def m4_proposed(n):
    # T(n) = 2T(n/2) + n, assuming T(1) = 1
    return n * (math.log2(n) + 1)
```

```
def m5_operations(n):
    # Recurrence-tree estimate for T(n)=2T(n/2)+n log2(n)
    log_n = math.log2(n)
    return n * log_n * (log_n + 1) / 2 + n
def question_d():
    print("\n========== (d) MASTER THEOREM ==========")
    print("\nM4 CURRENT:")
    print("a=4, b=2, f(n)=n")
    print("n^(log_b(a)) = n^2")
    print("Case 1: Theta(n^2)")
    print("\nM4 PROPOSED:")
    print("a=2, b=2, f(n)=n")
    print("n^(log_b(a)) = n")
    print("Case 2: Theta(n log n)")
    print("\nM5:")
    print("a=2, b=2, f(n)=n log n")
    print("n^(log_b(a)) = n")
    print("Extended Master Theorem Case 2")
    print("Complexity = Theta(n log^2 n)")
    print("\nEstimated recurrence costs:")
    print(f"M4 current: {m4_current(N):,.0f}")
    print(f"M4 proposed: {m4_proposed(N):,.0f}")
    print(f"M5: {m5_operations(N):,.0f}")
    print("\nWhen n doubles:")
    print("M4 current: approximately 4 times the work")
    print("M4 proposed: slightly more than 2 times the work")
# (e) Best, Average and Worst Search Cases
def question_e():
    print("\n========== (e) SEARCH CASES ==========")
    print("\nS1 - LINEAR SEARCH")
    print("Best: product is first = 1 comparison")
    print("Average successful = (n+1)/2 comparisons")
    print("Worst successful: product is last = n comparisons")
    print("Worst unsuccessful = n comparisons")
```

```
    p = 0.5
    average = N - p * (N - 1) / 2
    print("Expected comparisons = n - p(n-1)/2")
    print(f"For n={N:,}, p=0.5: {average:,.2f}")
    print("\nS2 - BINARY SEARCH")
    print("Best: middle product matches = 1 comparison")
    print("Average = Theta(log n)")
    print("Worst = Theta(log n)")
    print("For one million items: at most about 20 comparisons")
# (f) Prove Theta(n^2) and Not O(n)
def question_f():
    print("\n========== (f) ASYMPTOTIC BOUND ==========")
    print("f(n) = 3n^2 + 10n log2(n) + 50")
    print("Choose c1=1, c2=63, n0=1")
    n = 10
    f = 3 * n**2 + 10 * n * math.log2(n) + 50
    print(f"Example: f(10) = {f:.2f}")
    print("For n >= 1:")
    print("n^2 <= f(n) <= 63n^2")
    print("Therefore, f(n) = Theta(n^2)")
    print("\nTo prove f(n) is not O(n):")
    print("f(n) >= 3n^2, so f(n)/n >= 3n.")
    print("The ratio grows without bound.")
    print("Therefore, f(n) is not O(n).")
# (g) Scalability Decision
def estimate_time(operations):
    return operations / OPS_PER_SECOND
def show_estimate(name, operations):
    seconds = estimate_time(operations)
    print(f"{name:<26} {operations:>18,.2f} operations"
          f" {seconds:>12.4f} seconds")
```

```
def check_deadline(name, operations):
    seconds = estimate_time(operations)
    if seconds > DEADLINE:
        status = "EXCEEDS 10 MINUTES"
    else:
        status = "WITHIN 10 MINUTES"
    print(f"{name}: {status} ({seconds:.2f} seconds)")
def question_g():
    print("\n========== (g) SCALABILITY DECISION ==========")
    print(f"Input size = {N:,}")
    print(f"Assumed rate = {OPS_PER_SECOND:,} operations/second")
    print(f"Nightly limit = {DEADLINE} seconds\n")
    m1 = m1_comparisons(N)
    m2 = m2_audits(N)
    s2 = binary_search_comparisons(N)
    current = m4_current(N)
    proposed = m4_proposed(N)
    m5 = m5_operations(N)
    print("ESTIMATED OPERATIONS AND RUNNING TIMES:")
    show_estimate("M1 Duplicate Detector", m1)
    show_estimate("M2 API Rate Limiter", m2)
    show_estimate("S2 One binary search", s2)
    show_estimate("M4 Current Design", current)
    show_estimate("M4 Proposed Design", proposed)
    show_estimate("M5 Sales Analytics", m5)
    print("\nNIGHTLY DEADLINE CHECK:")
    check_deadline("M1", m1)
    check_deadline("M2", m2)
    check_deadline("M4 Current", current)
    check_deadline("M4 Proposed", proposed)
    check_deadline("M5", m5)
    print("\nFINAL RECOMMENDATIONS:")
    print("1. Redesign M1 using a hash-based duplicate detector.")
    print("2. Adopt the proposed M4 design.")
```

```
    print("3. Use binary search on the sorted product index.")
    print("4. Keep M2 and monitor performance.")
    print("5. Benchmark M5 using real data.")
    print("Note: Running times are estimates, not benchmarks.")

# Main program
if __name__ == "__main__":
    question_a()
    question_b()
    question_c()
    question_d()
    question_e()
    question_f()
    question_g()
```

Implementation:

```
========== (g) SCALABILITY DECISION ==========
Input size = 1,000,000
Assumed rate = 100,000,000 operations/second
Nightly limit = 600 seconds

ESTIMATED OPERATIONS AND RUNNING TIMES:
M1 Duplicate Detector      499,999,500,000.00 operations    4999.9950 seconds
M2 API Rate Limiter              1,048,575.00 operations       0.0105 seconds
S2 One binary search                  20.00 operations       0.0000 seconds
M4 Current Design          1,999,999,000,000.00 operations   19999.9900 seconds
M4 Proposed Design              20,931,568.57 operations       0.2093 seconds
M5 Sales Analytics             209,599,497.10 operations       2.0960 seconds

NIGHTLY DEADLINE CHECK:
M1: EXCEEDS 10 MINUTES (4999.99 seconds)
M2: WITHIN 10 MINUTES (0.01 seconds)
M4 Current: EXCEEDS 10 MINUTES (19999.99 seconds)
M4 Proposed: WITHIN 10 MINUTES (0.21 seconds)
M5: WITHIN 10 MINUTES (2.10 seconds)

FINAL RECOMMENDATIONS:
1. Redesign M1 using a hash-based duplicate detector.
2. Adopt the proposed M4 design.
3. Use binary search on the sorted product index.
4. Keep M2 and monitor performance.
5. Benchmark M5 using real data.
Note: Running times are estimates, not benchmarks.
```

```
========== (d) MASTER THEOREM ==========

M4 CURRENT:
a=4, b=2, f(n)=n
n^(log_b(a)) = n^2
Case 1: Theta(n^2)

M4 PROPOSED:
a=2, b=2, f(n)=n
n^(log_b(a)) = n
Case 2: Theta(n log n)

M5:
a=2, b=2, f(n)=n log n
n^(log_b(a)) = n
Extended Master Theorem Case 2
Complexity = Theta(n log^2 n)

Estimated recurrence costs:
M4 current: 1,999,999,000,000
M4 proposed: 20,931,569
M5: 209,599,497

When n doubles:
M4 current: approximately 4 times the work
M4 proposed: slightly more than 2 times the work
```

```
========== (c) BACKWARD SUBSTITUTION ==========
T(n) = T(n/2) + 1
T(n) = T(n/4) + 2
T(n) = T(n/8) + 3
After k steps: T(n) = T(n/2^k) + k
Stop when n/2^k = 1, so k = log2(n)
Since T(1) = 1, T(n) = 1 + log2(n)
Time complexity = Theta(log n)
Iterative binary search space = Theta(1)
Recursive binary search space = Theta(log n)
Comparisons for one search at n=1,000,000: 20
```

```
========== (a) UNDERSTAND THE PROBLEM ==========
M1: Pair comparisons - Theta(n^2)
M2: Request audits - Theta(n)
S1: Linear search - O(n)
S2: Binary search - Theta(log n)
M4 Current: 4T(n/2) + n - Theta(n^2)
M4 Proposed: 2T(n/2) + n - Theta(n log n)
M5: 2T(n/2) + n log n - Theta(n log^2 n)

========== (b) ITERATIVE MODULES ==========
M1 comparisons = n(n-1)/2
M1 time complexity = Theta(n^2)
M1 auxiliary space = Theta(1)
M1 comparisons at n=1,000,000: 499,999,500,000

M2 audit operations form a geometric series.
M2 time complexity = Theta(n), not Theta(n log n)
M2 auxiliary space = Theta(1)
M2 audits at n=1024: 1023
M2 audits at n=1,000,000: 1,048,575
```

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I **A. Mohith Krishna** (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **A. Mohith Krishna** Date: **11-10-26**